BLOCKCHAIN AT YONSEI

CONFIDENTIAL

BFT와 mysticeti v2



BAY 모진영

50, Yonsei-ro, Seodaemun-gu,
Seoul, Republic of Korea

https://www.blockchainatyonsei.com/

Contents

01 BFT 란?

02 Sui 컨센서스의 등장

03 1월 14일 네트
워크 중단 문제

01 BFT

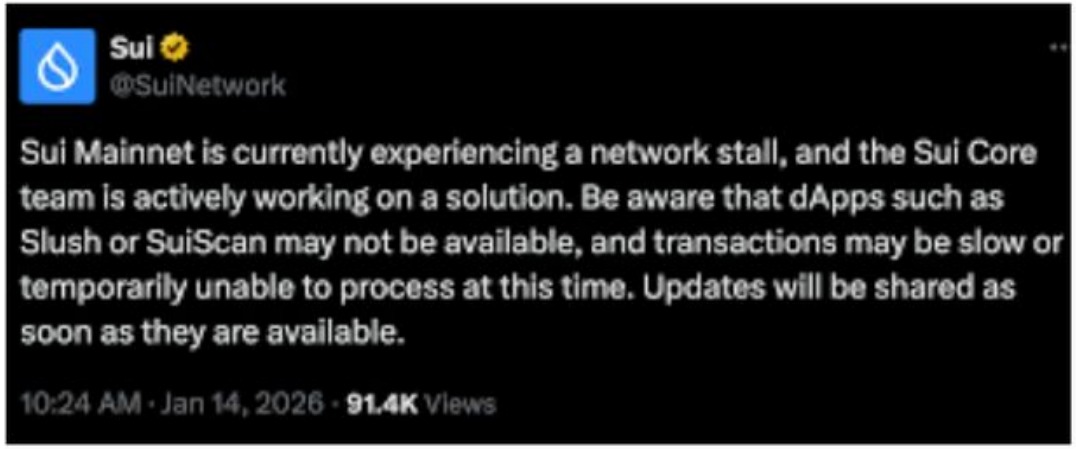
비잔틴 장군문제
Safety와 Liveness
FLP 불가능성 정리
Leader Based BFT
DAG Based BFT

02 Sui 컨센서스의 등장

Narwhal & BullShark
Mysticeti
Mysticeti v2

03 1월 14일 네트워크 중단 문제

개요



디지털 자산 · 이슈

수이 메인넷 6시간 멈춘 이유 밝혔다… “합의 오류·자산 피해 없었다”

양윤모 기자 • 2026-01-16 14:51 2 Mins Read

요약

수이 메인넷이 내부 합의 처리 오류로 약 6시간 등안 중단됐으나 검증인들의 신속한 대응으로 정상화됐다.

이번 오류는 트랜잭션 충돌에 대한 예외 처리 미흡이 원인이었으며, 사용자 자산에는 피해가 없었다.

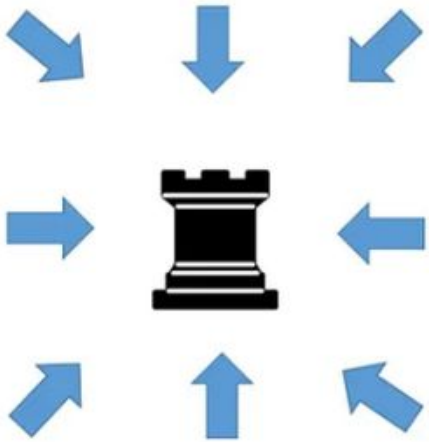
재단은 체크포인트 불일치 조기 감지 및 복구 자동화 등 재발 방지책을 강화할 계획이다.

BLOCKCHAIN AT YONSEI

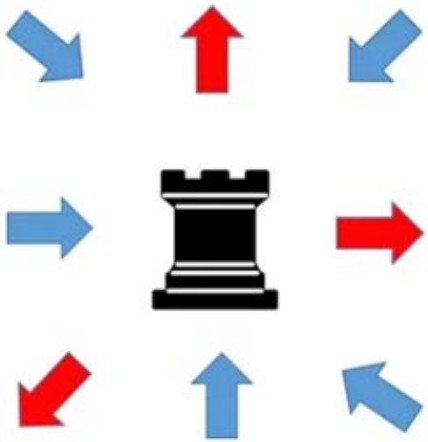
BFT

50, Yonsei-ro, Seodaemun-gu,
Seoul, Republic of Korea

비잔틴 장군 문제(Byzantine Generals Problem)



Victory



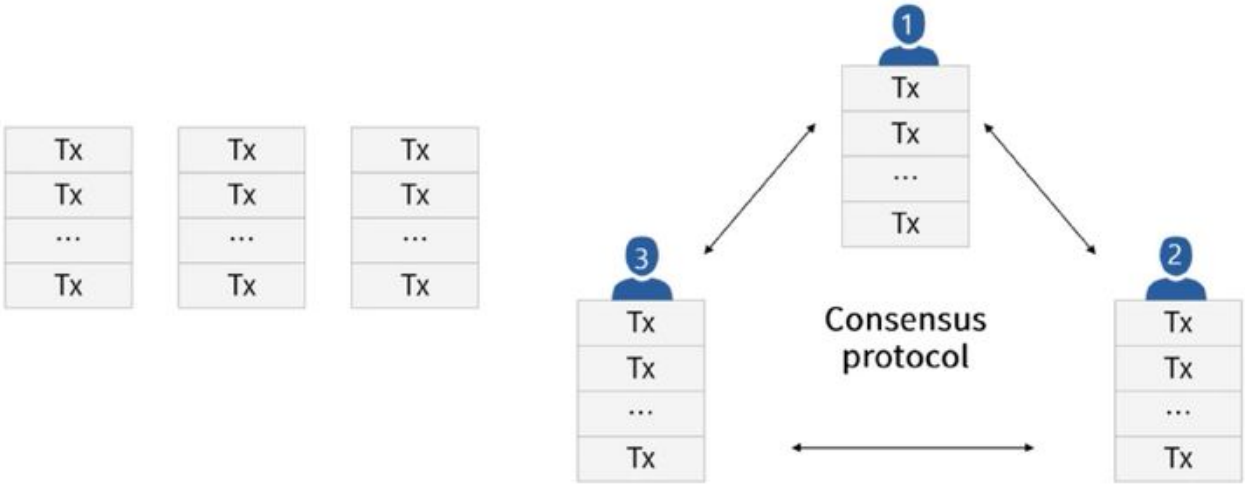
Defeat

The Byzantine Generals Problem에서 처음 제시된 컴퓨터 공학의 난제

1. 장군들 사이에는 적과 내통하는 배신자가 존재하고, 모든 장군들은 해당 배신자가 누군지는 모르나 배신자가 존재한다는 사실을 모두 알고 있다.
2. 충성스러운 장군들이 계획에 맞춰 한꺼번에 도시를 공격한다면, 함락에 성공한다.
3. 공격 계획에 대한 장군들 간의 소통은 전령(messenger)에 의해서만 이루어질 수 있다.

https://medium.com/@organmo/%EB%B8%94%EB%A1%9D%EC%B2%B4%EC%9D%B8-%ED%95%A9%EC%9D%98-%EC%95%8C%EA%B3%A0%EB%A6%AC%EC%A6%98-%EB%B6%84%EC%84%9D-%EC%BD%94%EC%8A%A4%EB%AA%A8%EC%8A%A4%EC%9D%98-tendermint-3-64f5c74b7e8c

비잔틴 장군 문제(Byzantine Generals Problem)



블록체인도 결국 분산시스템... 트랜잭션 순서라는 “단일 값“에 대한 합의가 필요함.
즉 Byzantine Fault Tolerance 가 필요하기에 BFT 합의 프로토콜이 필요!!

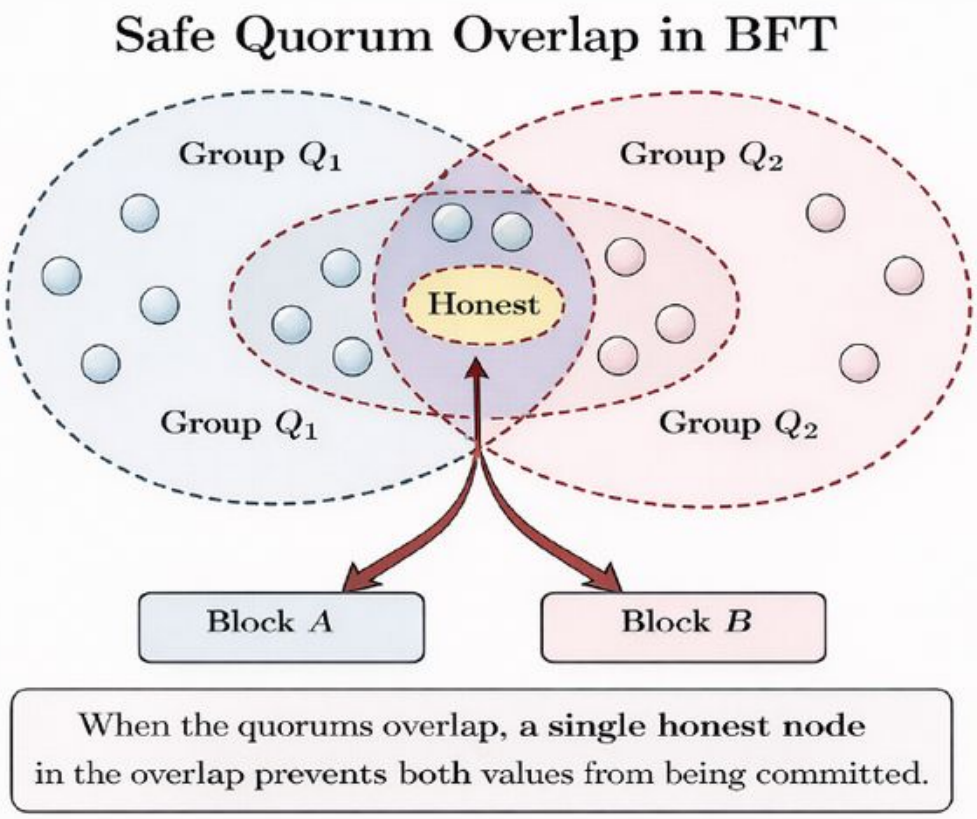
https://jjeongil.tistory.com/1707

Safety and Liveness

속성	정의	직관적 설명	위반 시 결과
Safety (안전성)	나쁜 일이 절대 일어나지 않음	"두 번 결정하지 않는다" — 합의된 것은 뒤집히지 않음	이중 지불, 포크 → 치명적
Liveness (생존성)	좋은 일이 결국 일어남	"언젠가 합의에 도달한다" — 시스템이 멈추지 않음	네트워크 정지 → 불편하지만 자금 안전

https://jeongil.tistory.com/1707

Safety



"Quorum이란 **어떤 결정을 승인하기 위해 필요한 최소 노드 집합 크기**

두 quorum이 겹치면, 그 안에 honest 노드가 하나라도 있으
그 노드는 두 값에 동시에 서명 할 수 없음.

Lemma: Why $|Q_1 \cap Q_2| \geq 2q - n$? (Pigeonhole Principle)

BLOCKCHAIN AT YONSEI

Setup

Total nodes: n (labeled $1, 2, \ldots, n$)



Two quorums, each of size q

Q_1 (size q) Q_2 (size q)

Step 1: Minimum Overlap

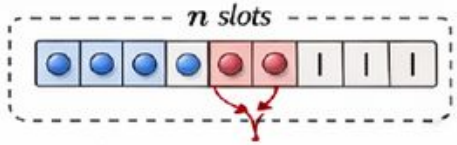
Selecting $2q$ nodes from n positions → overlap is inevitable!



Max non-overlap $= n - q$

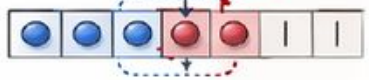
$\Rightarrow$ Minimum overlap $= q - (n - q) =$ **$2q - n$**

Intuition (Pigeonhole Principle)



Place $2q$ people into n slots → some slots get filled twice!

$\Rightarrow$ At least $2q - n$ overlaps



Step 2: Byzantine Constraint

⚠ Up to f Byzantine nodes in $Q_1 \cap Q_2$



To guarantee ≥ 1 honest node in overlap:

$$2q - n \geq f + 1$$

Step 3: Safety Condition

Combine Lemma + Byzantine constraint

$$|Q_1 \cap Q_2| \geq 2q - n$$

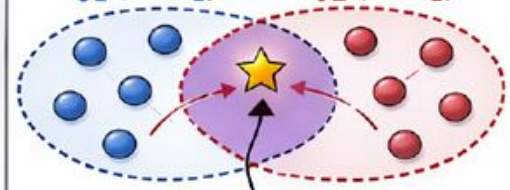
$$2q - n \geq f + 1$$

$$\Rightarrow \quad q \geq \frac{n + f + 1}{2}$$

✓ Safety requires this quorum size!

Visual Summary

Q_1 (size q) Q_2 (size q)



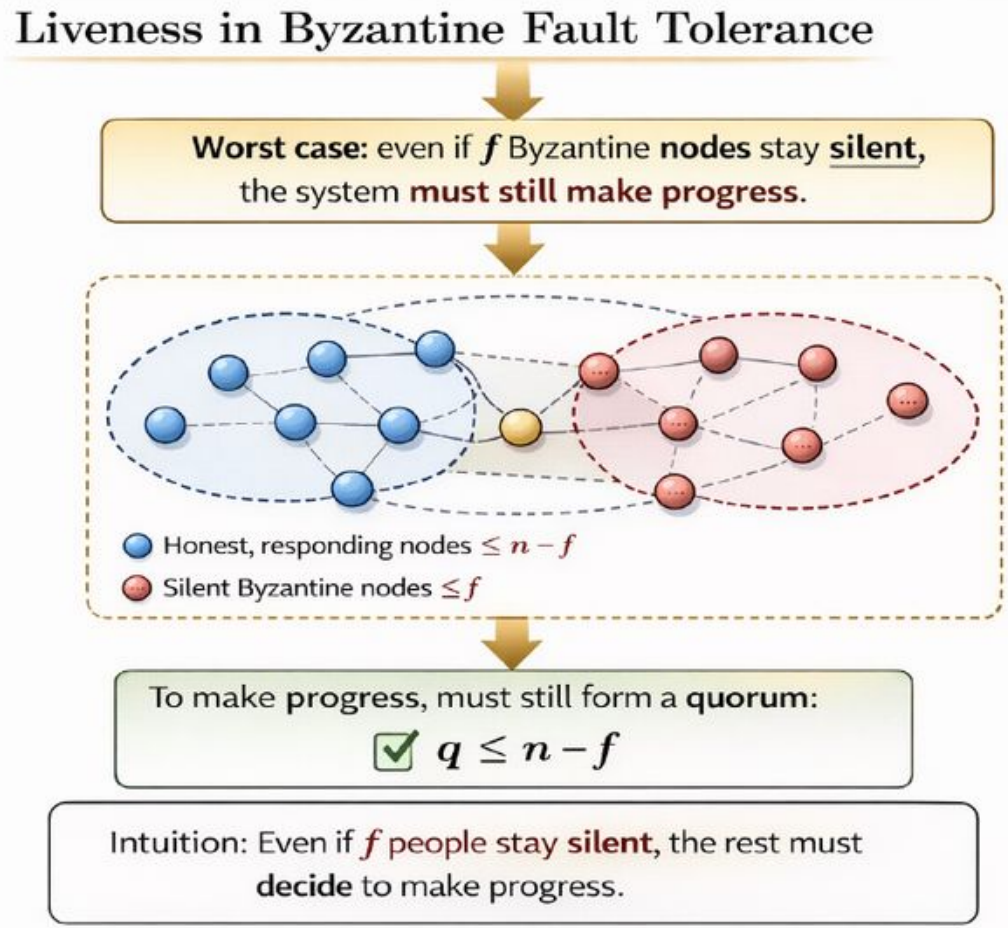
★ ≥ 1 Honest (if $2q - n \geq f + 1$)

Key Takeaways

Key Takeaways

1. Two quorums (g each) **overlap** by $\geq 2q - n$
2. Need $\geq f + 1$ in overlap $\Rightarrow 2q - n \geq f + 1$
3. ☑ Final **Safety Bound:** $q > \frac{n + f + 1}{2}$

Liveness



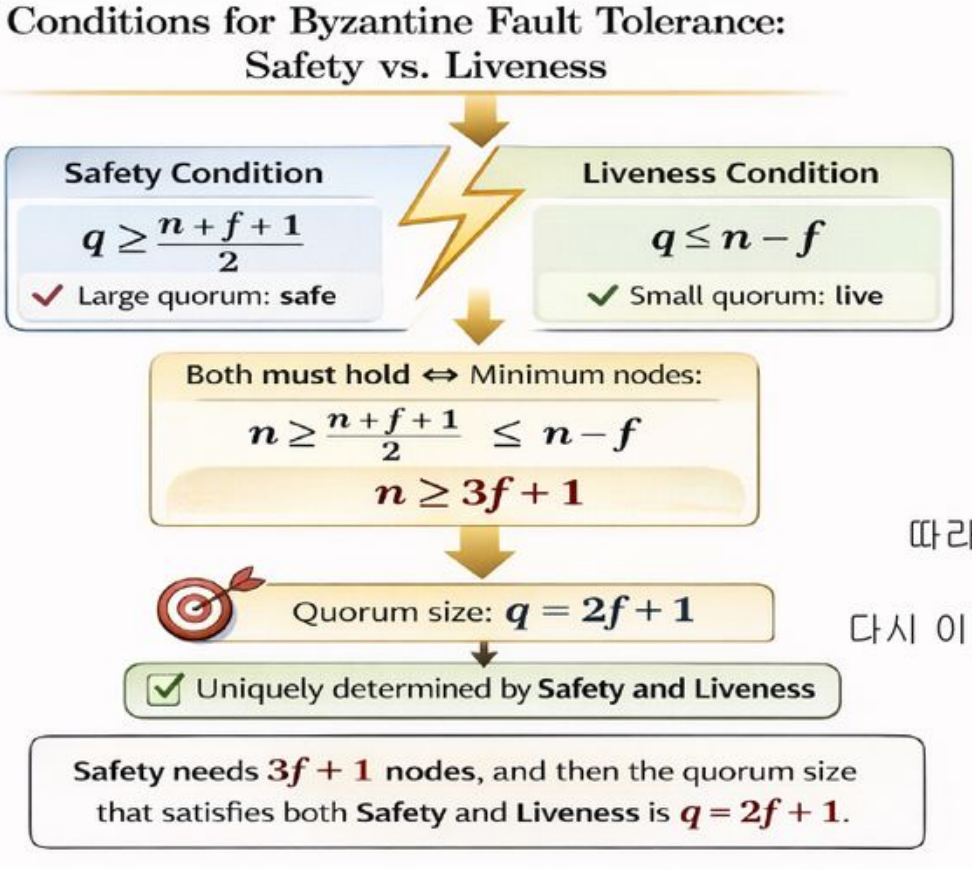
여기서 비잔틴 노드(f)는 Safety와 달리 침묵하는 노드라고 가정.

전체 노드가 n이라고 하면
응답 가능한 최대 노드는 $n - f$

결국엔 응답가능 n-f 으로도 쿼럼을 채울 수 있어야한다.
따라서 Livness Condition은

$$q \leq n - f$$

Safety and Liveness



아까 구한 Safety Condition과 Liveness Condition을 합하면

$$q \geq \frac{n+f+1}{2} \qquad q \leq n-f$$

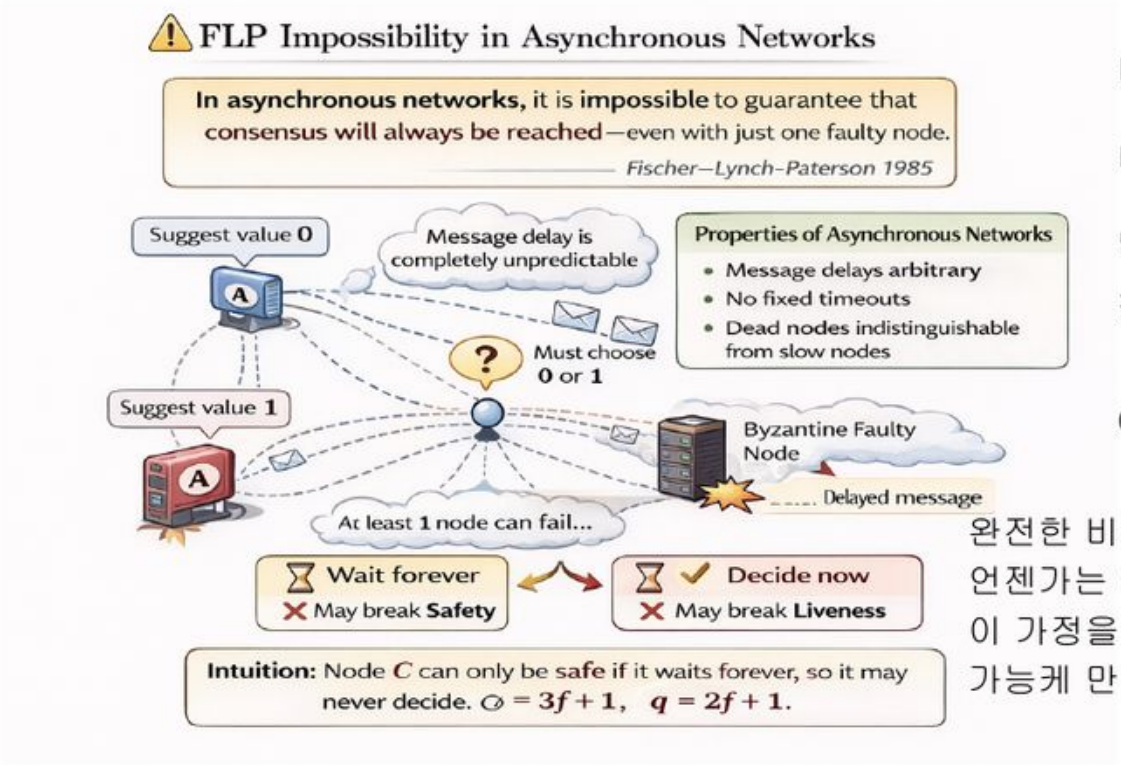
$$\frac{n+f+1}{2} \leq n-f$$

$$n \geq 3f+1$$

따라서 비잔틴을 견디기 위한 최소 노드 수는 $n = 3f+1$

다시 이 식을 Safety 조건식에 넣으면 최소 쿼럼(정족 수)는 $q = 2f+1$

FLP 불가능성 정리(FLP Impossibility Theorem)



비동기 네트워크에서는 메시지 지연 시간이 무한히 길어질 수 있음

따라서 느린 노드와 고장 난 노드를 구분할 방법이 없음

단 1개의 노드만 실패(Crash) 해도,

결정을 내리면 → 나중에 모순이 생길 수 있고 (Safety 위험)
기다리면 → 영원히 결정 못 할 수도 있음 (Liveness 실패)

어떤 알고리즘도 Safety와 Liveness를 동시에 항상 보장할 수 없다.

완전한 비동기 환경에서는 합의는 원래 불가능 그래서 실제 BFT 시스템은 언젠가는 네트워크가 안정된다는 가정 (Partial Synchrony)을 함
이 가정을 통해 타임아웃을 "증명 가능한 판단" 으로 바꿔서 Liveness를 가능케 만듬.

FLP 불가능성 정리(FLP Impossibility Theorem)



나카모토 계열 컨센서스들..

우리가 익히 알고있는 비트코인이 획기적인 이유는 Pow로 이걸 해결하였기 때문인데
Liveness를 보장해서 Safety를 확률적으로 점진 보장 하였기 때문이다..

6블록 지나서 Finalize 취급한다는게 Safety 보장의 의미로 이해하면 됨.



BFT 계열의 컨센서스들..

PBFT의 후손들... HotStuff, DAG 등등 각자의 방식으로 진화함

Safety를 항상 보장하고 부분적 동기를 통해 Liveness도 보장한다. (Safety가 우선)

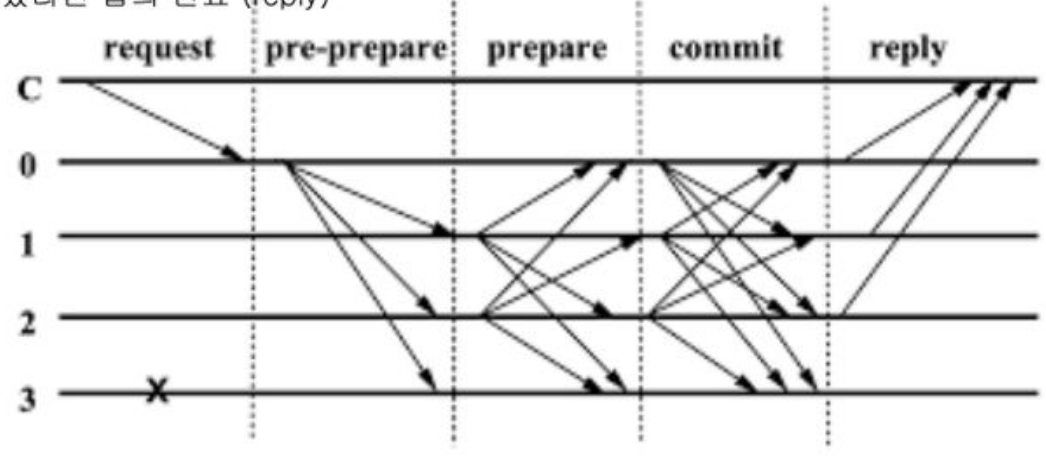
완전히 비동기적인 환경에서는 메시지가 영원히 지연될 수 있기 때문에 합의가 멈출 수 있다.
이를 극복하기 위해 실제 시스템은 언젠가는 메시지 지연이 유한해진다(Eventual Synchrony)고 가정하고,
타임아웃과 리더 교체를 반복하여 결국 합의가 진행되도록 만든다.

Leader Based BFT

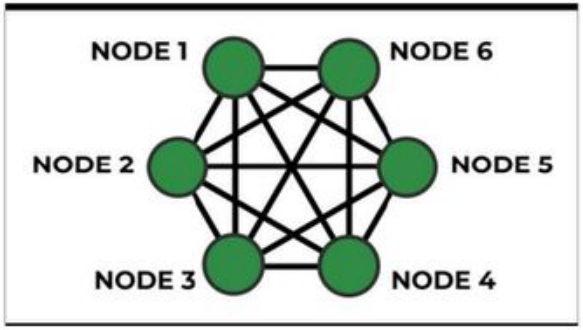
- 트랜잭션 순서 합의를 위해 Replica 의 역할이 다음과 같이 나누어짐
  - 하나의 Replica 는 임의의 방식으로 Leader 가 되어 임의의 트랜잭션 순서를 "제안"함
  - 나머지 Replica 는 Leader 가 제안한 트랜잭션 순서에 "투표"

PBFT - Practical Byzantine Fault Tolerance(최초의 실용적 비잔틴 장애 허용)

1. 리더인 Replica 0 이 Client 의 Request 를 보고 다른 Replica 에게 제안 (pre-prepare)
2. 나머지 Replica 는 해당 제안에 대한 투표를 다른 Replica 에게 제안 (prepare)
3. 각 Replica 는 과반수이상의 투표 수신 여부를 다른 Replica 에게 전파 (commit)
4. Commit Phase 에서 과반수 이상의 Replica 로부터 올바른 응답을 받았다면 합의 완료 (reply)



Leader Based BFT - PBFT



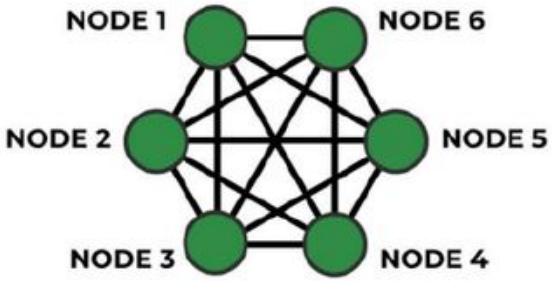
- 통신 복잡도가 $O(n^2)$ (all-to-all broadcast)
- 리더 노드의 부재시 신규 리더 노드 선출에 시간 복잡도가 매우 높음
- Prepare 단계: 리더를 제외한 모든 노드가 다른 모든 노드에게 prepare 메시지 브로드 캐스팅
- Commit 단계: 각 노드가 다른 노드들로부터 충분한 수 (2f + 1)의 prepare 메시지를 받으면, commit 메시지를 다른 모든 노드에게 브로드캐스팅

- 소규모의 permissioned 환경의 체인에서는 PBFT는 충분히 매력적이다.
  (그래서 실제로 하이퍼레저 계열은 PBFT를 사용했음)
- 그러나 Open 된 생태계에서는 속도 문제로 인한 어려움 존재

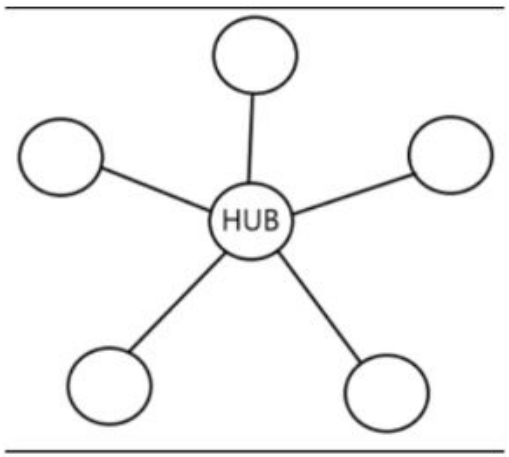
- 최악의 경우 리더 교체(View Change)시 $O(n^3)$ 가됨.



Leader Based BFT - HotStuff



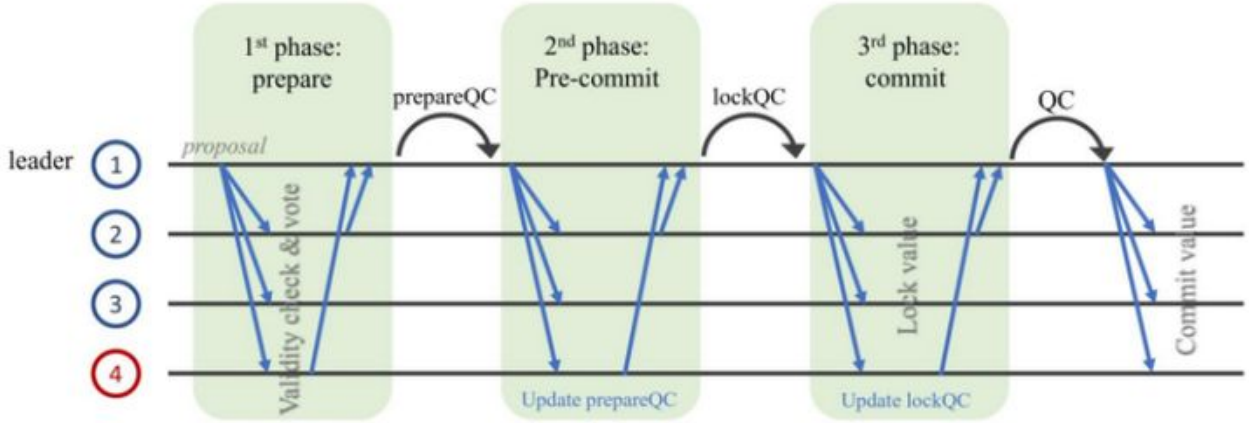
- PBFT 와 같은 All - All 전파 방식이 아닌
  Leader를 중심으로 하는 메시지 전파
- 통신 복잡도를 $O(n^2)$에서 → $O(n)$으로 줄이게 되는 계기
- QC (Quorum Certificate, 쿼럼 인증서)가 핫스터프의 핵심적인 구조가 됨



- 개별 노드의 서명들을 합친 thresholding signature 즉 2f + 1개 이상의 replica들이 투표했음을 증명하게됨

Leader Based BFT - HotStuff

Basic HotStuff



- prepareQC : qualified certificate for $2f + 1$ validity votes.
- lockQC: qualified certificate for $2f + 1$ locks on a proposal.

Leader:

- 이전 종료시점에 레플리카 들이 보낸 new-view 메시지를 수집하며 기다리다가 (어느 QC까지 간거냐?) 체크
- 가장 높은 번호 QC를 선택해서 Proposal 생성함
- 그리고 전파.....

Replica:

- 리더로 부터 prepare 메시지 대기하다가
- 리더의 제안이 올바른지 확인
- 확인되면 Vote 메시지 보냄...

- 이렇게 3 phase 진행해서 확정. 리더 집중식으로 된것만 빼면 이론적 근거는 PBFT를 계승함

Leader Based BFT - HotStuff

Chained Hotstuff

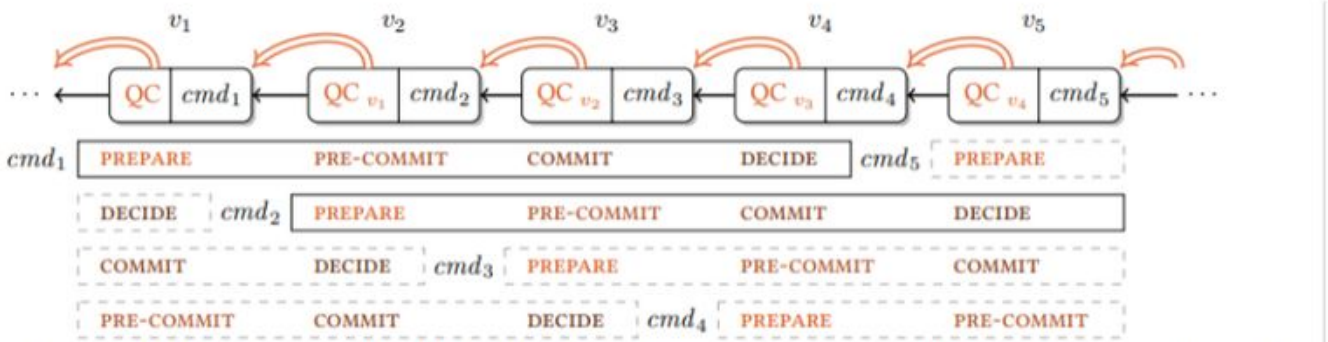


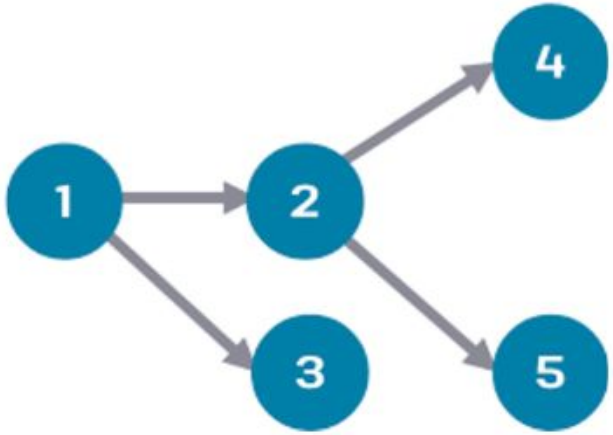
Figure 1: Chained HotStuff is a pipelined Basic HotStuff where a QC can serve in different phases simultaneously.

- 이렇게 3 Phase 합의 절차를, 동시에 겹쳐서(pipelining) 비동기적으로 흘려보내는 구조
- PBFT 의 시간 순서로 진행하는 동기적 블록생성 절차(한 블록 = 3라운드 동기 절차)를 비동기 파이프라인으로 만듬
- 연속된 블록이 서로의 합의 단계를 수행한다고 보면됨 → 3-chain commit rule 조건을 통해서

$B_1 \leftarrow B_2 \leftarrow B_3$ QC가 3개 연속으로 연결되면 가장 앞 블록이 확정됨.

DAG Based BFT

DAG(Directed Acyclic Graph)

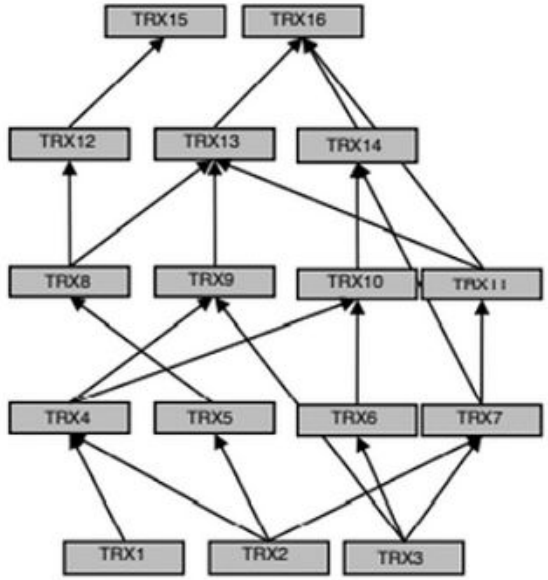


- 방향 비순환 그래프 라는 자료 구조로
- 그래프가 Vertex(정)과 그 점들을 연결하는 Edge(간선)으로 이루어진 구조임

Blockchain



Directed Acyclic Graph



DAG Based BFT

- 기존의 PBFT와 HotStuff는 모두 리더가 제안해서 나머지가 투표하는 구조를 따름
- HotStuff가 PBFT의 시간 복잡도를 $O(n^2)$ → $O(n)$으로 해결했으나 리더가 모든 데이터를 중계하는 병목이 생김

리더 한명이 제안하는 구조 자체(즉 리더 방식)을 없앨 수 있을까? 모든 노드가 데이터를 제안하는 구조??

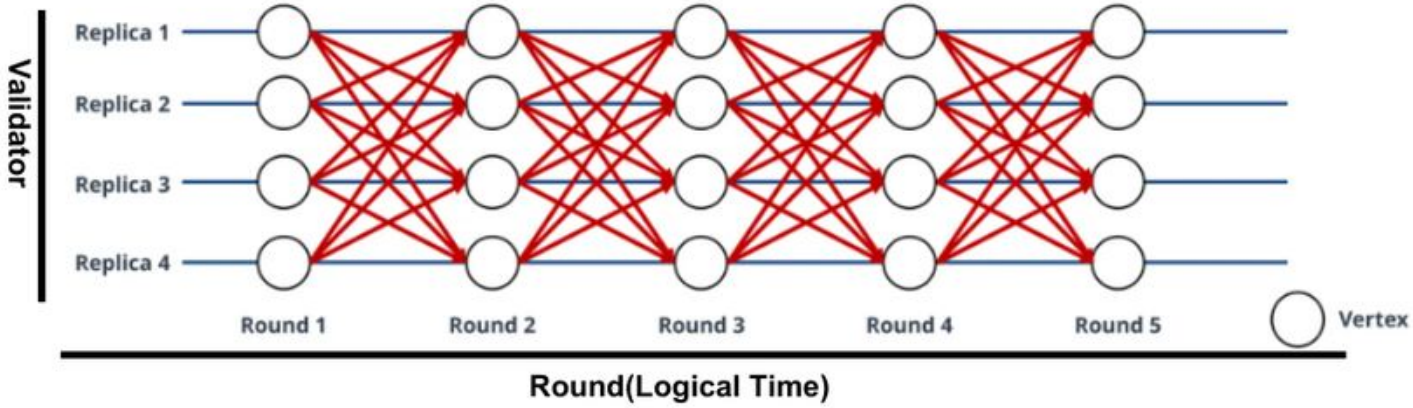
DAG-based BFT는 선형 체인(Linear Chain)에서 DAG(Directed Acyclic Graph)로 데이터 구조를 전환함으로써, 합의의 패러다임 자체를 교체함

(1) 모든 노드가 리더

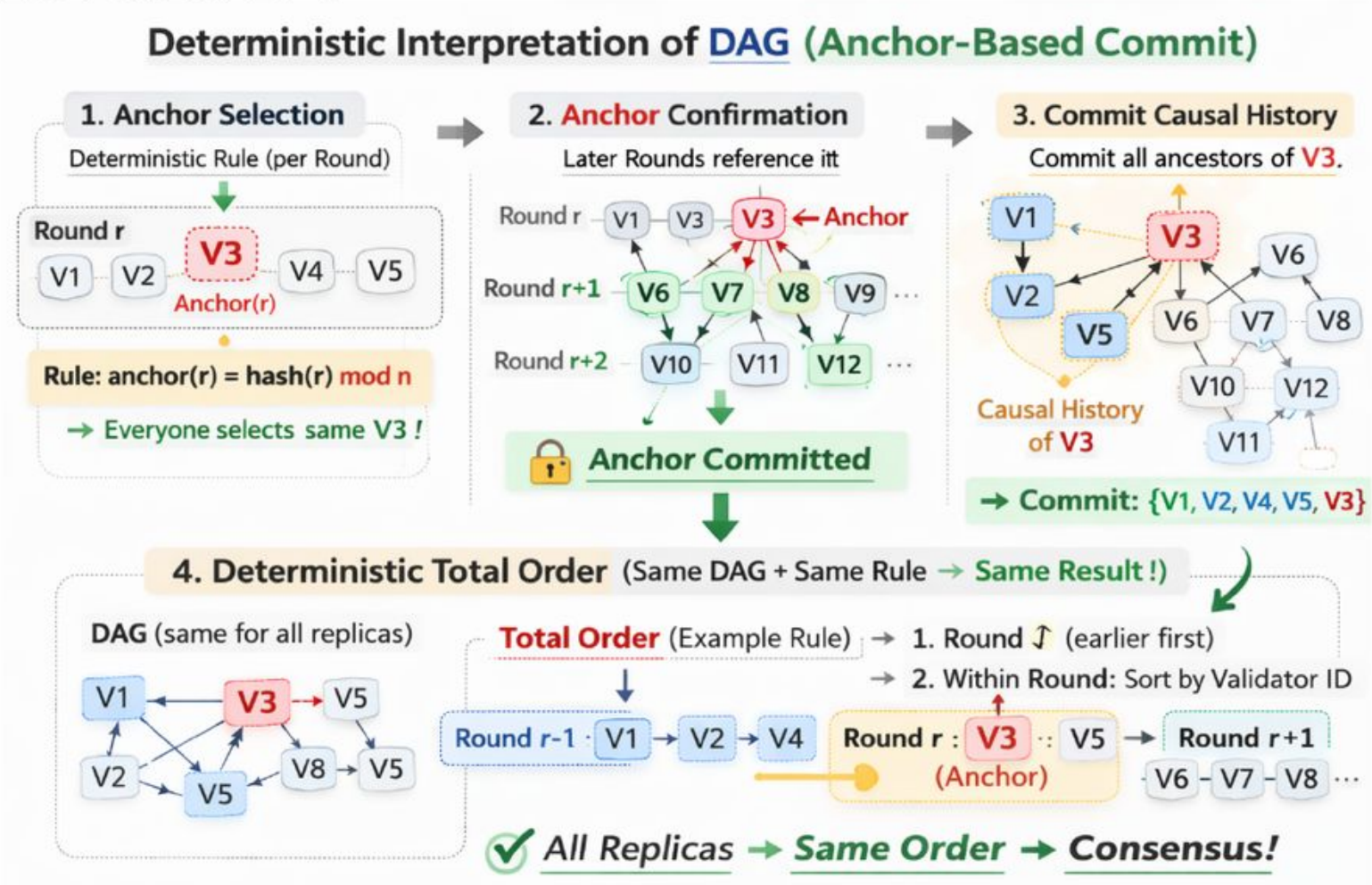
(2) Vertex와 Edge 투표 없는 합의

(3) 로컬 DAG 해석으로 전역 합의 도달

(4) 합의 메시지 오버헤드 = 0



DAG Based BFT



BLOCKCHAIN AT YONSEI

Sui 컨센서스의 등장

Narwhal & BullShark

기존의 컨센서스는 데이터 전파와 순서 결정을 하나의 프로토콜에서 처리....

→ 그걸 분리해보면 어떨까???...

데이터 전파와 데이터 저장 증명

순서를 결정



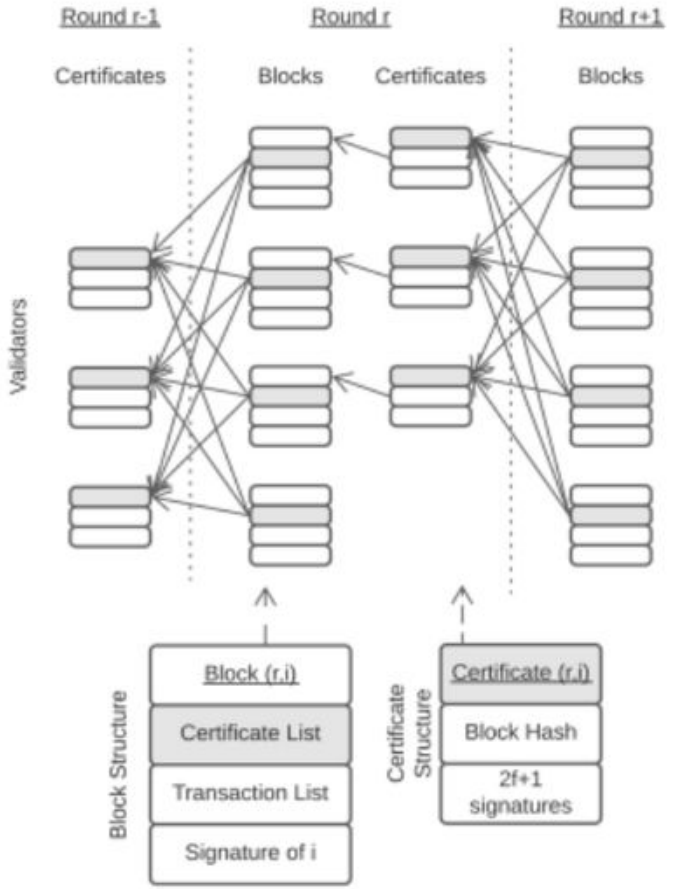
(Narwhal)



(bullshark)

Narwhal

Narwhal → 데이터 가용성 레이어

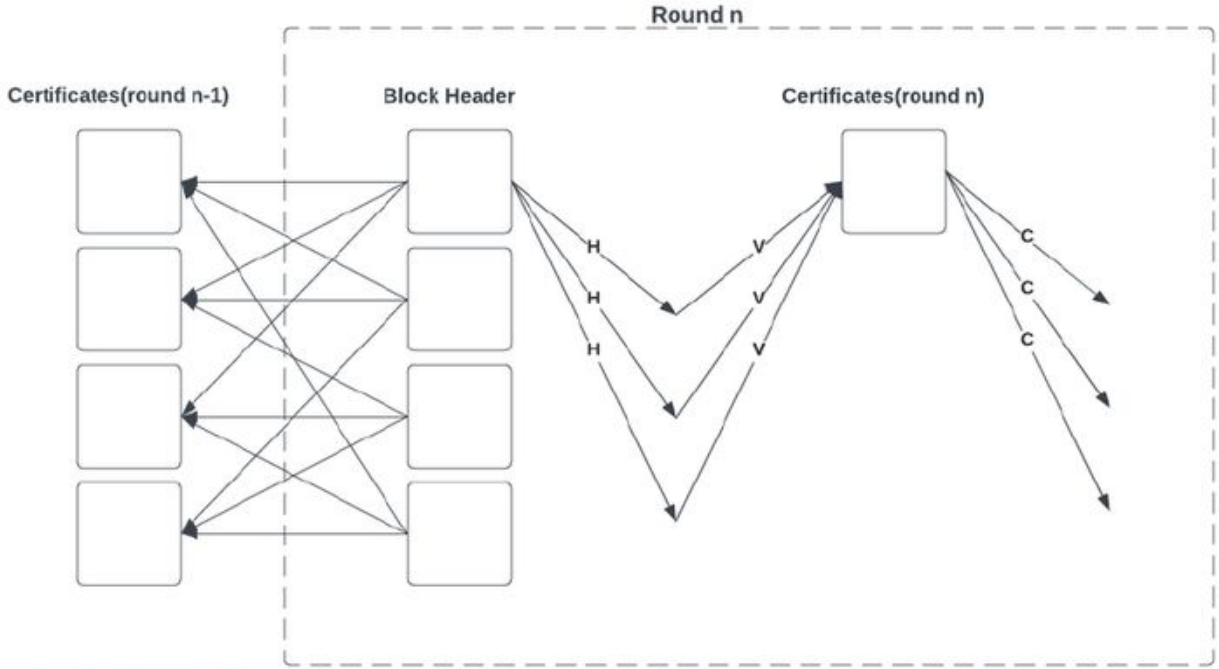


- Worker: 트랜잭션 배치(~500KB) → 병렬 전파
- Primary: 배치 해시(32B)만 수집 → DAG Vertex 생성
- Certificate: 2f+1 서명 = "이 데이터가 존재한다"는 증명

Worker 추가 → 처리량 선형 증가 (상한 없음)



Narwhal



- Certificates(r-1) 참조 → Block Header 생성
- H: 헤더를 다른 Primary에게 전송
- V: 2f+1 서명(Vote) 수집
- C: 인증서 완성 → 다음 라운드 입력

라운드 r의 정점은 반드시 라운드 r-1의 정점 2f+1개를 참조!!!!

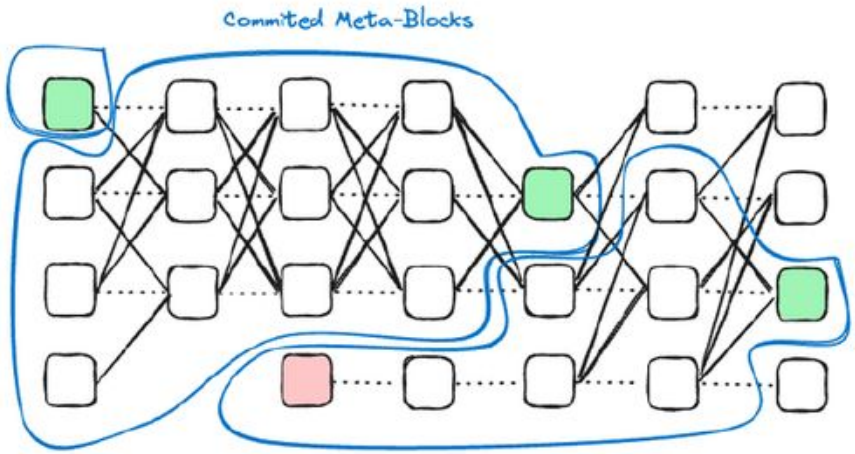
2f+1개를 참조했다는 것은 해당 데이터가 최소 f+1개의 정직한 노드에 존재함

왜??? 2f + 1 의 정족수가 모였을 때 그 중 f개가 비잔틴이면 f + 1

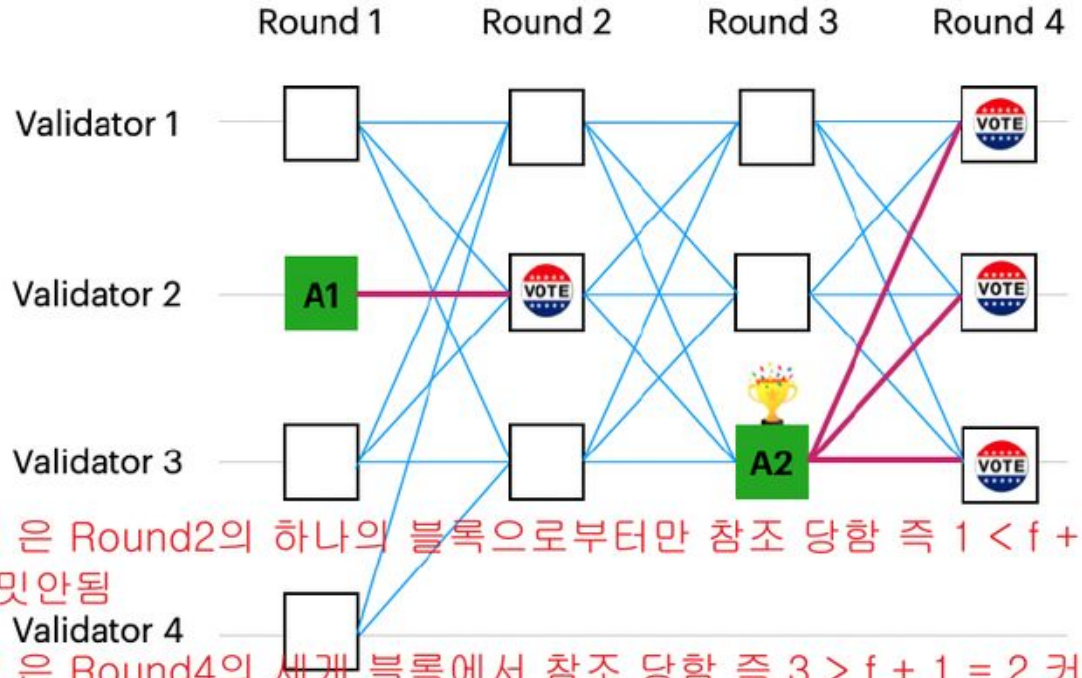
i.e 2f + 1 − f = f + 1

나할이 만든 DAG를 불샤크가 해석하는데 1홉을 직접참조, 2홉 이상을 간접참조라고 해석함.

BullShark - Zero message overhead consensus



초록색 블록이 커밋될 때 파란색으로 묶인 노드집합이 한번에 커밋됨



A1 은 Round2의 하나의 블록으로부터만 참조 당함 즉 1 < f + 1 = 2 커밋안됨

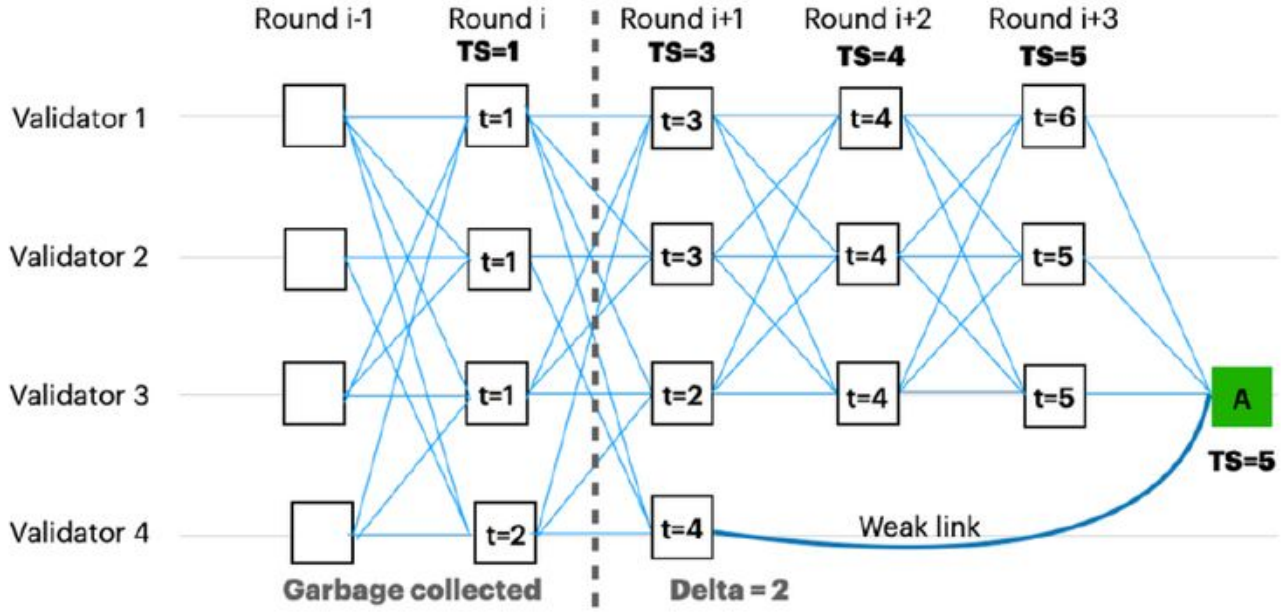
A2 은 Round4의 세개 블록에서 참조 당함 즉 3 > f + 1 = 2 커밋

이미 Narwhal이 DAG를 구축하면서 데이터 가용성 + 인과관계 보장

나할이 만들어놓은 DAG 로컬에서 해석만 하자 !!!!

이렇게만 해도 전역적 순서 결정 가능!

Garbage Collection

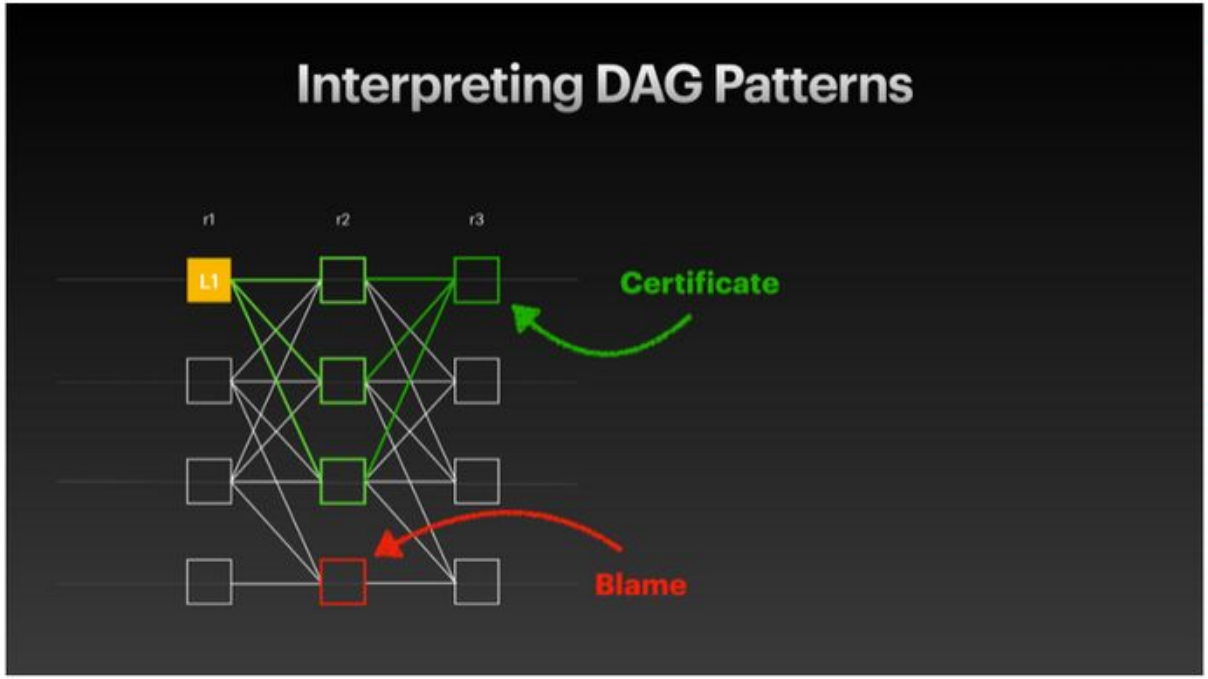


DAG는 매 라운드마다 n개의 블록이 추가됨....

무한히 저장할 수 없으므로 이미 커밋된 과거 라운드의 데이터를 삭제(GC, Garbage Collection)해야 한다..

따라서 이 GC를 하는 시점은 모든 노드가 동일해야한다!!!! 비동기 환경에서는 각 노드가 DAG를 보는 시점이 다른게 문제... → 바로 이 문제가 이번 Sui 컨센서스가 멈춘이유..

Mysticeti



r1: 리더 L1이 블록 제안 (서명 포함, 즉시 전파)

r2: 2f+1 블록이 L1 참조 = 투표 (초록)

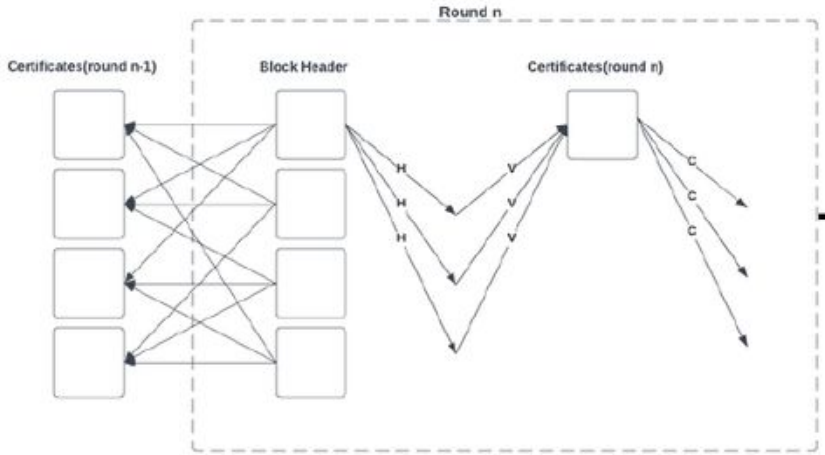
r3: 2f+1 블록이 투표 포함 = 인증서 역할 → 커밋!

다시 말해서 r2에서 벨리 A, B, C가 L1을 참조
하는게 투표가 되고 또 그 3개의 투표를 본
r3에서의 밸리 D가 그 투표가 존재한다는걸 검증!

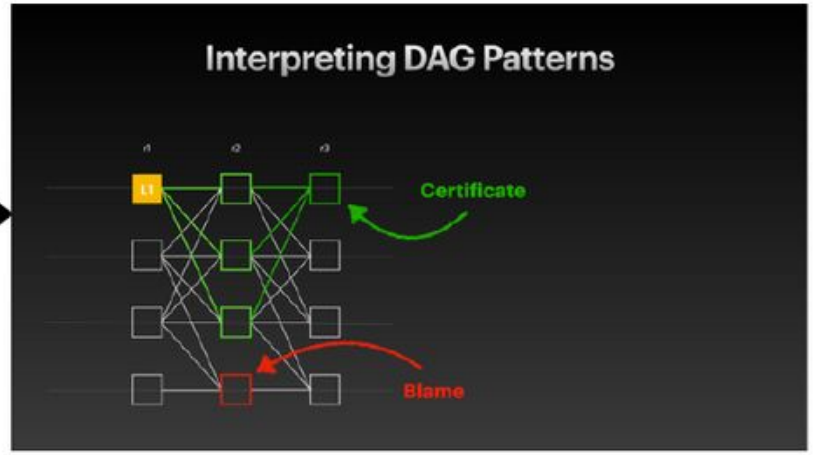
Narwhal이 하던 일(DAG 구축)과 Bullshark가 하던 일(합의 해석)을 모두 새로 설계

Mysticeti

사전인증

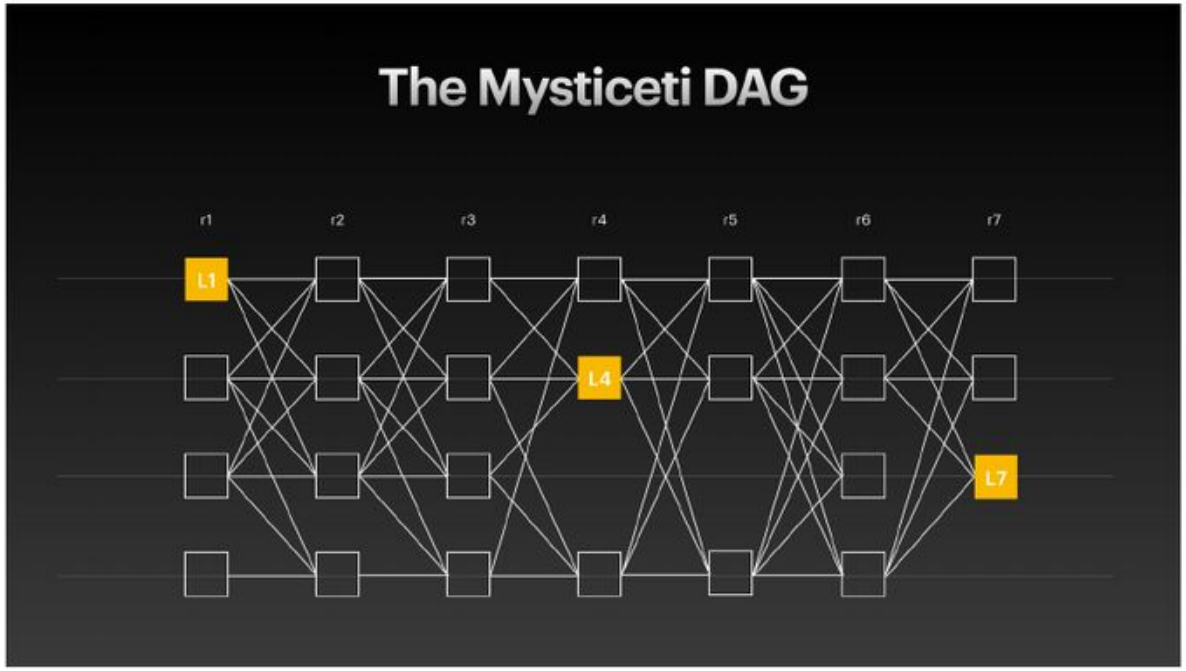


사후인증



다시 말해서 기존에 있던 H → V → C 라운드 자체를 없앴것이다.

Mysticeti



Bullshark: 앵커 1명 실패 → 해당 라운드 전체 지연

Mysticeti: 모든 블록이 잠재적 앵커
→ 크래시에도 성능 유지!!

Bullshark는 짝수 라운드에만 앵커 1명이었는데, Mysticeti는 매 라운드마다 리더가 있고, 모든 블록이 커밋 가능!!

Mysticeti - Mysticeti-FPC 소유 객체 Fast Path

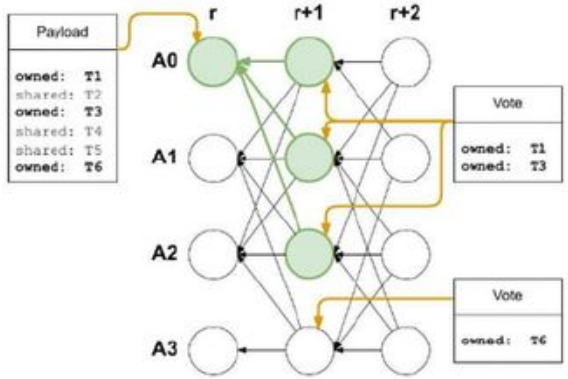


Fig. 8: Illustration fast path transaction execution. The blocks $(A_0, r, \cdot)$ contain the fast path transactions T_1, T_3, and T_6. Blocks $(A_0, r+1, \cdot), (A_1, r+1, \cdot), (A_2, r+1, \cdot)$ support (A_0, r, L_r) and explicitly vote for T_1 and T_3 (but not T_6). Upon observing these blocks, the validator can safely execute T_1 and T_3.

소유 객체는 합의 안한다....

Sui Move 로 개발하신분은 아시겠지만 Owned, Shared Object 라는 특이한 개념이있다.. (ownership model)

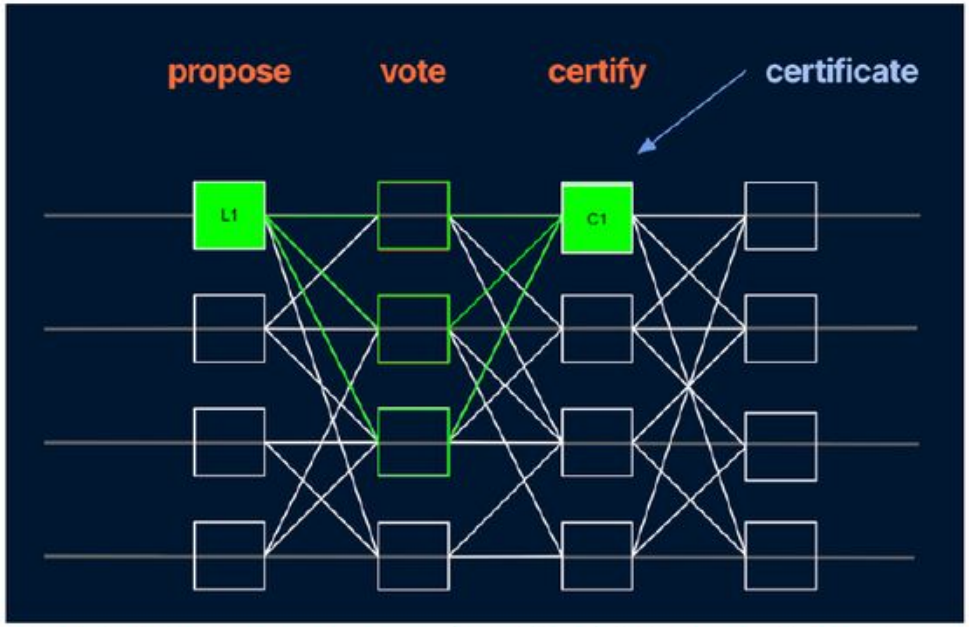
내가 소유한 즉 Owned 객체(내 코인,NFT)를 내가 보내겠다는데 누가 반대하겠는가?

비트코인의 UTXO랑 비슷하다..

```
// 이더리움: 계정 기반
Account 0x1234 → balance: 10 ETH  (하나의 숫자)

// Sui: 객체 기반
Coin<SUI> 0xAAA → value: 5 SUI   (객체 1)
Coin<SUI> 0xBBB → value: 3 SUI   (객체 2)
Coin<SUI> 0xCCC → value: 2 SUI   (객체 3)
```

Mysticeti v2





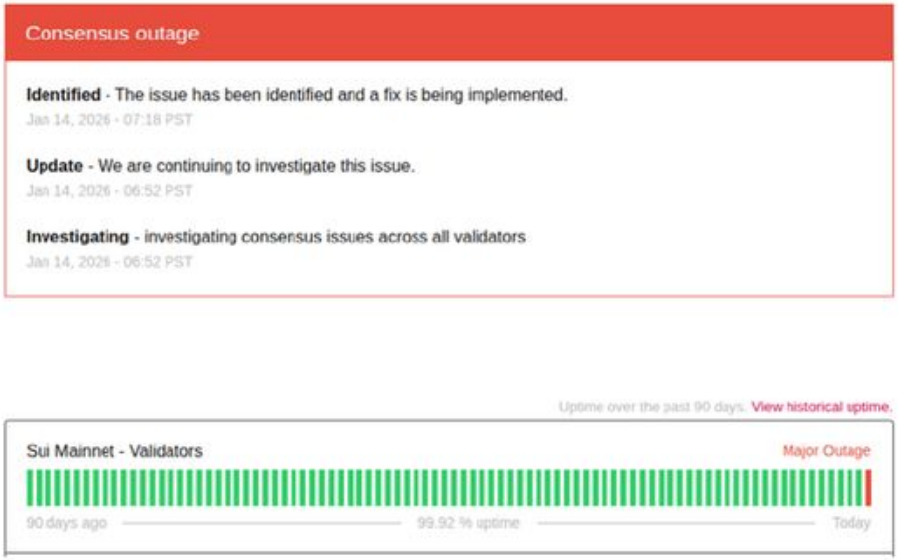
합의 전(Pre-consensus)에 별도 검증 단계 존재 → 그것도 합의 엔진에 넣음...
그럼 왜 애초부터 별도 검증 단계가 합의엔진 통합이 안되있던 거지? 라는 의문...

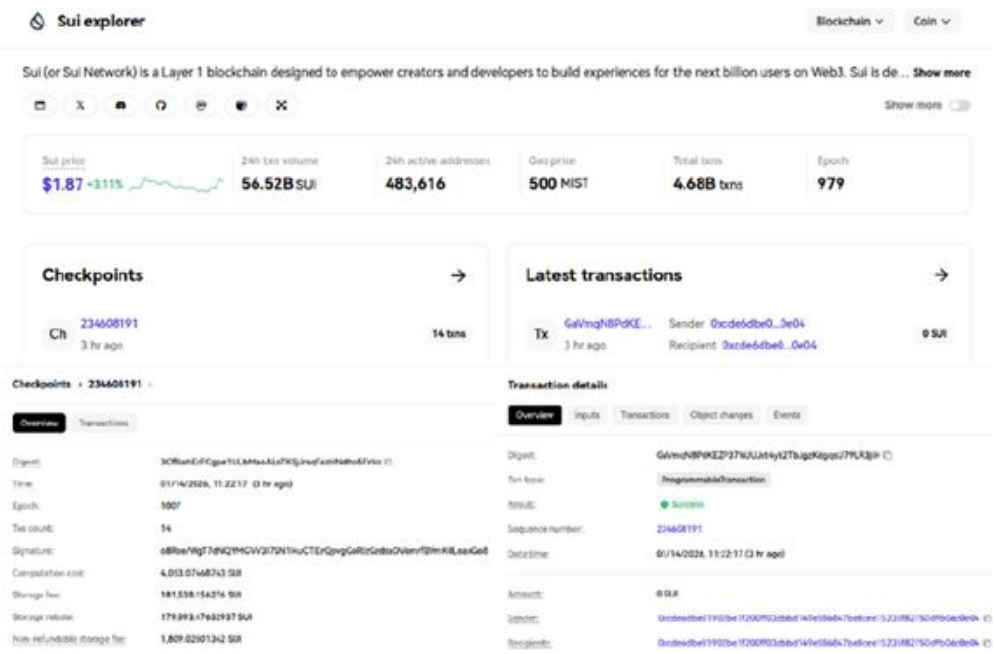
결국엔 레거시(Narwhal/Bullshark) 때문 저 불샤크 모델을 쓸 때도 수이팀은 소유객체를 합의 안하고 싶은 욕심을 부림 → 그래서 Quorum Driver 라는 트랜잭션 검증 드라이버를 두었었는데 그게 공유 객체에도 적용이 되있던 것.
(공유 객체는 어차피 합의 엔진을 거치는 데 말이다)

BLOCKCHAIN AT YONSEI

1월 14일 네트워크 중단 문제

1월 14일 네트워크 중단 문제



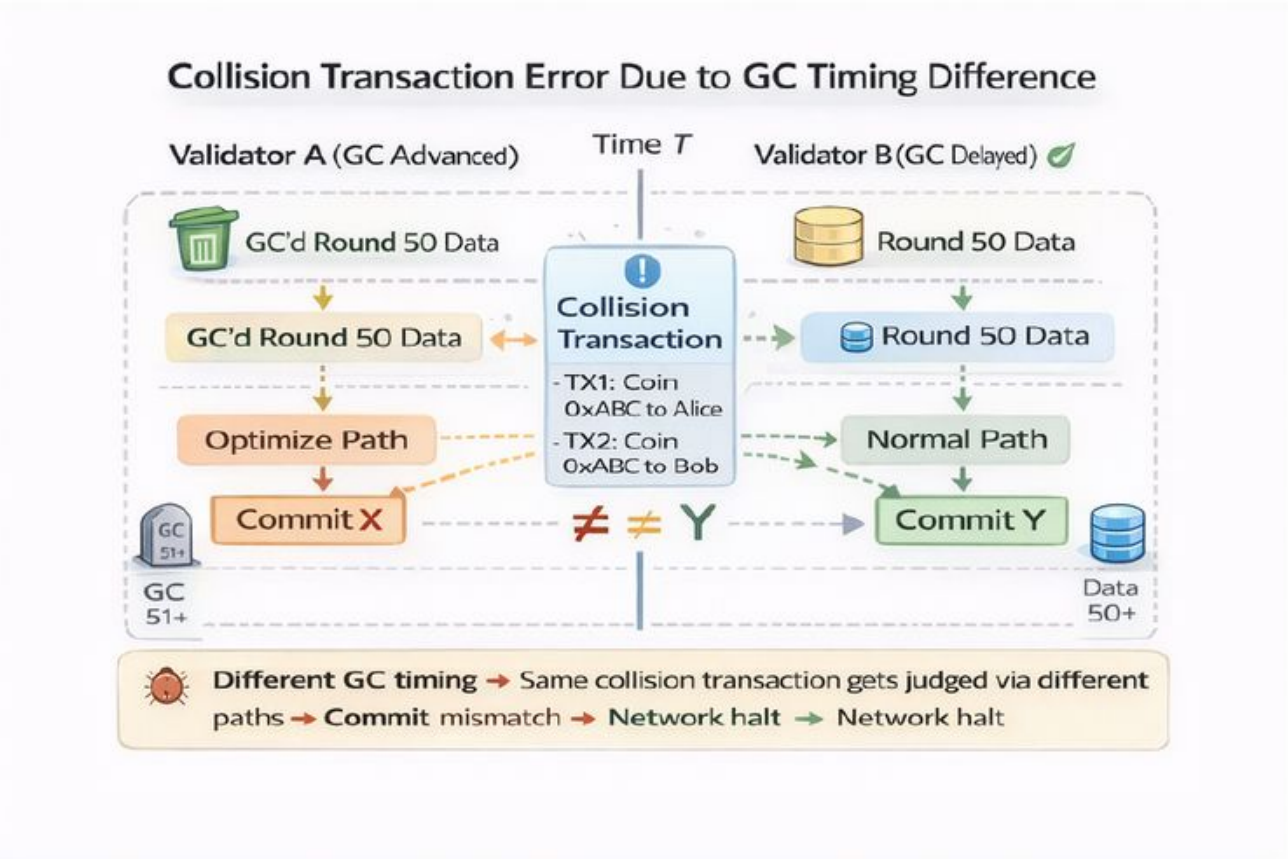


순수 엣지케이스로 인한 로직버그..

1) 충돌 트랜잭션 발생
Tx1: 코인 0xABC를 Alice에게 전송
Tx2: 코인 0xABC를 Bob에게 전송 같은코인임...
이때 합의가 TX1을 선택하고, TX2는 버려야

→ 하지만 이 때 그 블록이 GC로 사라진 벨리에서 다른 방식으로 판단을 내림

1월 14일 네트워크 중단 문제



BLOCKCHAIN AT YONSEI

CONFIDENTIAL

Thank You

2026.02.19

모진영



50, Yonsei-ro, Seodaemun-gu,
Seoul, Republic of Korea

https://www.blockchainatyonsei.com/